
Popularity-Aware LightGCN for Mitigating Popularity Bias in Graph-Based Recommender Systems

Nguyen Quoc Hung Bui Trung Kien Nguyen Minh Thuong
Nguyen Nam Hoang Nguyen Tuan Kiet

School of Information and Communication Technology
Hanoi University of Science and Technology

Hung.NQ252568M@sis.hust.edu.vn, Kien.BT252567M@sis.hust.edu.vn
Thuong.NM252280M@sis.hust.edu.vn, Hoang.NN252758M@sis.hust.edu.vn
Kiet.NT252763M@sis.hust.edu.vn

Abstract

Graph-based recommender systems can capture high-order collaborative signals, but may amplify popularity bias toward high-degree items. We study this problem on MovieLens 1M by formulating movie recommendation as an implicit-feedback user-item graph. We propose Popularity-Aware LightGCN, which combines Item Loss Equalization, degree-aware graph augmentation, contrastive regularization, and popularity-aware negative sampling. The method is evaluated using chronological splitting and full-catalog ranking with both accuracy and popularity-aware metrics. Compared with LightGCN, the balanced configuration increases TailRecall@20 by approximately $6.5\times$ and Catalog Coverage@20 by 35%, while slightly improving Recall@20. These results show that popularity-aware training can substantially improve long-tail recommendation behavior without sacrificing overall ranking accuracy. The implementation and datasets used in this study are publicly available in the GitHub repository: https://github.com/TrungKien/Graph_ML.git

1 Introduction

Recommender systems help users discover relevant items from large catalogs. In this project, we study movie recommendation using the MovieLens 1M dataset [1], where user-movie interactions are represented as a bipartite graph. The task is to generate top- K movie recommendations for each user. Beyond standard recommendation accuracy, we focus on popularity bias, where highly popular movies dominate recommendation lists while long-tail movies receive limited exposure. To address this issue, we compare standard recommendation baselines with popularity-aware LightGCN variants [2] that incorporate loss equalization, degree-aware graph augmentation, contrastive regularization, and popularity-aware negative sampling.

1.1 Motivation

Movie recommendation data is usually highly imbalanced: a small number of popular movies receive many interactions, while many less popular movies receive limited feedback. This long-tail structure can cause recommender systems to over-recommend head items and under-expose middle or tail items [3]. Although graph-based models such as LightGCN are effective at capturing collaborative signals from the user-movie graph, their neighborhood aggregation process may also amplify the influence of high-degree items. As a result, a model can achieve strong Recall@ K or NDCG@ K while still producing recommendation lists that are heavily concentrated on popular movies. Therefore, this project evaluates both recommendation accuracy and long-tail recommendation behavior.

1.2 Problem statement

Given MovieLens 1M, we convert explicit ratings into implicit positive feedback by treating ratings greater than or equal to 4 as positive interactions. These interactions define a user-movie bipartite graph $\mathcal{G} = (\mathcal{U} \cup \mathcal{I}, \mathcal{E})$, where $\mathcal{U}$ is the set of users, $\mathcal{I}$ is the set of movies, and each edge $(u, i) \in \mathcal{E}$ indicates that user u positively interacted with movie i . For each user u , a model assigns a relevance score $s(u, i)$ to each candidate movie i that does not appear in the user’s training interactions. Movies are then ranked by these scores, and the top- K unseen movies are returned as recommendations.

The objective of this project is twofold: to achieve accurate top- K recommendation and to analyze whether the model reduces over-concentration on popular movies. We compare MostPopular, BPR-MF, and LightGCN with popularity-aware LightGCN variants. These variants introduce training-time mechanisms at different levels, including the objective, graph structure, representation space, and negative sampling distribution. The main research question is whether these strategies can improve the exposure and retrieval of less popular movies while maintaining competitive recommendation accuracy.

1.3 Contributions

The main contributions of this project are summarized as follows:

- We formulate MovieLens 1M as a graph-based top- K recommendation task by converting explicit ratings into implicit feedback and constructing a user-movie bipartite graph.
- We implement and compare MostPopular, BPR-MF, and LightGCN to analyze popularity-based ranking, personalized matrix factorization, and graph-based collaborative filtering.
- We investigate popularity-aware LightGCN variants that combine loss equalization, degree-aware graph augmentation, contrastive regularization, and popularity-aware negative sampling to mitigate popularity bias.
- We evaluate the implemented models using a unified full-ranking pipeline with both accuracy metrics and popularity-bias metrics, including Recall@K, NDCG@K, TailRecall@K, Catalog Coverage@K, Average Recommendation Popularity, and head/middle/tail exposure.

As an overview, Figure 1 summarizes the workflow of this project, from data preprocessing and graph construction to model comparison and evaluation.

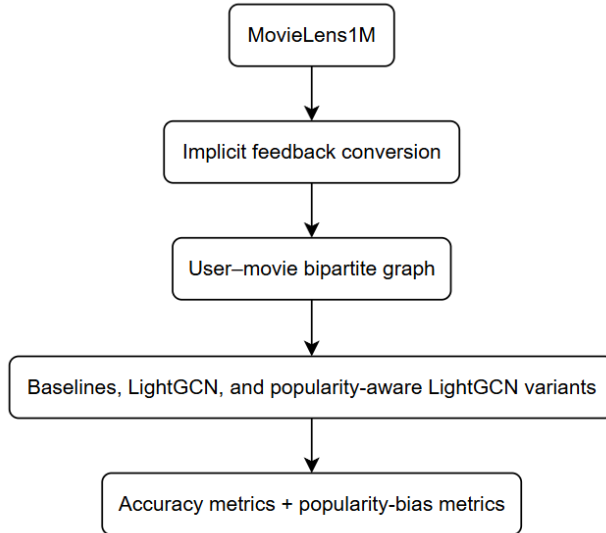


Figure 1: Overall project pipeline. MovieLens 1M is converted into implicit feedback and represented as a user-movie bipartite graph. The resulting graph is used to compare recommendation baselines and popularity-aware LightGCN variants under both accuracy and popularity-bias metrics.

2 Related work

This section reviews the main research directions related to our study. We first discuss implicit-feedback recommendation and graph-based collaborative filtering. We then summarize popularity bias and long-tail evaluation. Finally, we review popularity-aware learning strategies related to the proposed extensions of LightGCN.

2.1 Implicit-feedback and graph-based recommendation

Implicit-feedback recommendation learns user preferences from observed interactions rather than explicit preference values. In this project, MovieLens 1M ratings greater than or equal to 4 are treated as positive interactions, and the task is formulated as top- K ranking over unseen movies.

Bayesian Personalized Ranking (BPR) is a widely used objective for implicit-feedback recommendation [4]. Given a user, an observed positive item, and an unobserved negative item, BPR trains the model to rank the positive item higher. BPR-MF applies this objective to user and item embeddings and serves as a personalized non-graph baseline in our experiments.

User-item interactions can also be represented as a bipartite graph, enabling models to capture higher-order collaborative signals. LightGCN simplifies graph convolution for recommendation by retaining only normalized neighborhood aggregation and layer-wise embedding combination [2]. It is used as our main graph-based baseline. However, high-degree items participate in more interactions and message-passing paths, which may increase their influence on learned representations and amplify popularity bias.

2.2 Popularity bias and long-tail recommendation

Popularity bias occurs when recommender systems disproportionately favor highly popular items over less popular alternatives [3]. Recommendation data commonly follows a long-tail distribution, in which a small number of head items receive many interactions while most middle and tail items receive substantially less feedback. Consequently, models optimized primarily for ranking accuracy may repeatedly recommend popular items, reducing catalog diversity and the exposure of potentially relevant long-tail items.

Standard metrics such as Recall@ K and NDCG@ K measure ranking accuracy but do not fully capture this concentration. We therefore complement them with TailRecall@ K , Catalog Coverage@ K , Average Recommendation Popularity, and head/middle/tail exposure. Together, these metrics evaluate whether improvements in accuracy are achieved without excessively concentrating recommendations on head items.

2.3 Popularity-aware learning

Popularity bias can be addressed at different stages of model training. Item Loss Equalization (ILE) modifies the objective by penalizing discrepancies between losses associated with different popularity groups [5]. In our method, the losses of head and tail positive items are compared so that the model allocates more learning capacity to underrepresented tail interactions.

Graph augmentation and contrastive learning provide complementary structural and representation-level interventions. Self-supervised Graph Learning constructs perturbed views of the interaction graph and encourages the same node to maintain consistent representations across views [6]. Our degree-aware augmentation drops edges connected to popular items more aggressively and uses the resulting views in a contrastive objective, reducing the dominance of high-degree items during representation learning.

Negative sampling also affects the ranking signals received during BPR optimization. Uniform sampling may not sufficiently penalize popular items that dominate recommendation lists. We therefore use popularity-aware negative sampling, which selects high-degree items more frequently as negatives and discourages them from receiving uniformly high scores across users. The proposed Popularity-Aware LightGCN combines these objective-, structure-, representation-, and sampling-level strategies within a unified training framework.

Table 1: Raw MovieLens 1M statistics before preprocessing.

Statistic	Value
Users	6,040
Movies	3,883
Explicit ratings	1,000,209
Minimum interactions per user	20
Median interactions per user	96
Maximum interactions per user	2,314
Minimum interactions per movie	1
Median interactions per movie	123.5
Maximum interactions per movie	3,428

3 Dataset and graph learning task

This section describes the data source, preprocessing pipeline, graph construction, and learning task used in our popularity-aware movie recommendation project. The goal is to convert the original MovieLens 1M rating data into an implicit-feedback user-item graph that can be used consistently by non-graph baselines, matrix-factorization models, and graph neural recommenders such as LightGCN.

3.1 MovieLens 1M dataset

We use the MovieLens 1M dataset, which contains historical movie ratings from a large set of users. In the raw version loaded through PyTorch Geometric, the dataset contains 6,040 users, 3,883 movies, and 1,000,209 explicit rating interactions. Each interaction is represented as a tuple

$$(u, i, r_{ui}, t_{ui}), \quad (1)$$

where u is a user, i is a movie, $r_{ui} \in \{1, 2, 3, 4, 5\}$ is the explicit rating, and t_{ui} is the timestamp of the rating. The timestamp is important because it allows the train/validation/test split to respect the chronological order of each user’s behavior.

The raw rating distribution is highly concentrated around medium and high scores: there are 56,174 ratings of 1; 107,557 ratings of 2; 261,197 ratings of 3; 348,971 ratings of 4 and 226,310 ratings of 5. This distribution suggests that many observed interactions are positive or near-positive, but low ratings should not be treated as evidence of user preference. Therefore, before constructing the recommendation graph, we convert explicit ratings into implicit positive feedback.

3.2 Implicit feedback conversion and filtering

Top- K recommendation is usually formulated as a ranking problem over items that a user has not yet consumed. For this reason, our models do not predict the exact rating value. Instead, they learn from positive user-movie interactions. We convert the explicit rating matrix into an implicit-feedback dataset by keeping only ratings greater than or equal to 4:

$$y_{ui} = \begin{cases} 1, & r_{ui} \geq 4, \\ 0, & \text{otherwise.} \end{cases} \quad (2)$$

Only interactions with $y_{ui} = 1$ are retained as observed positive edges. This threshold keeps movies that users likely enjoyed while discarding weak or negative preference signals. After applying this conversion, the dataset contains 575,281 positive interactions.

We then remove users with fewer than five positive interactions. This filtering step ensures that every remaining user has enough history to support a per-user chronological split: at least one validation item, one test item, and several training interactions. After filtering, the dataset contains 6,034 users and 3,533 movies. We re-index all remaining user and movie identifiers into contiguous integer ranges so that they can be used directly as embedding-table indices in PyTorch and PyTorch Geometric.

Table 2: Statistics after implicit-feedback conversion, filtering, and chronological splitting.

Statistic	Value
Users	6,034
Items	3,533
Positive interactions	575,272
Training interactions	563,204
Validation interactions	6,034
Test interactions	6,034
Average positive interactions per user	95.34
Average positive interactions per item	162.83

3.3 Chronological train/validation/test split

For each user, interactions are sorted by timestamp. The most recent positive interaction is assigned to the test set, the second most recent interaction is assigned to the validation set, and all earlier positive interactions are assigned to the training set. Formally, for a user u with chronologically ordered positive items $(i_1, i_2, \dots, i_{n_u})$, we define

$$\mathcal{D}_{\text{test}}(u) = \{(u, i_{n_u})\}, \quad (3)$$

$$\mathcal{D}_{\text{val}}(u) = \{(u, i_{n_u-1})\}, \quad (4)$$

$$\mathcal{D}_{\text{train}}(u) = \{(u, i_1), \dots, (u, i_{n_u-2})\}. \quad (5)$$

This split is more realistic than a random split because it evaluates whether the model can recommend future items from past behavior. It also prevents information leakage from future interactions into the training graph.

The final preprocessed dataset contains 575,272 positive interactions: 563,204 training interactions, 6,034 validation interactions, and 6,034 test interactions. Each user contributes exactly one validation item and one test item. On average, the processed dataset has 95.34 positive interactions per user and 162.83 positive interactions per item.

3.4 User-movie bipartite graph construction

The recommendation data is naturally represented as a bipartite graph. Let $\mathcal{U}$ be the set of users and $\mathcal{I}$ be the set of movies. We construct a graph

$$\mathcal{G} = (\mathcal{V}, \mathcal{E}), \quad \mathcal{V} = \mathcal{U} \cup \mathcal{I}, \quad (6)$$

where an edge $(u, i) \in \mathcal{E}$ exists if user u has a positive training interaction with movie i . Only training interactions are used to build the graph. Validation and test interactions are held out exclusively for model selection and final evaluation.

Because LightGCN performs message passing over an undirected user-item graph, each positive training pair is represented by two directed edges: one from the user node to the movie node and one from the movie node back to the user node. Movie node indices are shifted by the number of users so that user and movie nodes occupy a single shared node-index space. With 563,204 training interactions, the resulting training graph has an edge-index tensor of shape $(2, 1126408)$, corresponding to both directions of all training edges.

This graph representation supports collaborative filtering through neighborhood aggregation. A user representation can receive information from movies the user liked, while a movie representation can receive information from users who liked that movie. After multiple propagation layers, LightGCN can capture higher-order collaborative signals such as users connected through similar movies and movies connected through similar audiences.

3.5 Popularity groups and long-tail structure

A central concern of this project is popularity bias. In recommendation data, items are not observed uniformly: a small number of popular movies often receive many interactions, while many long-tail

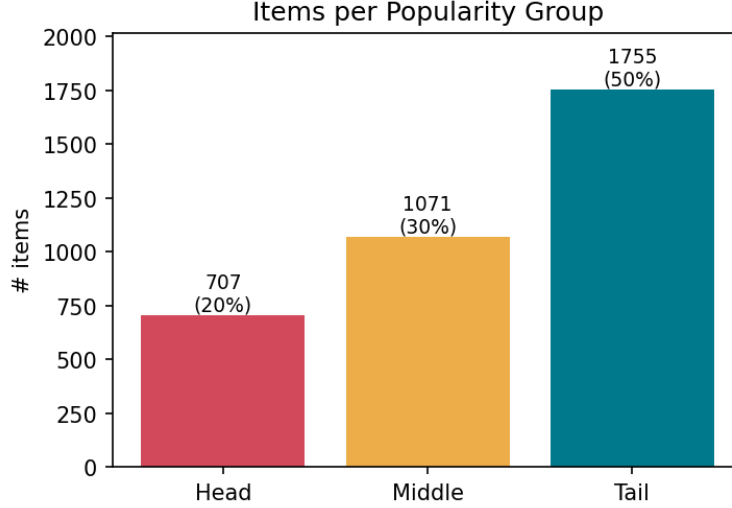


Figure 2: Number of movies in each training-degree popularity group. The updated preprocessing uses bottom 50% as tail, next 30% as middle, and top 20% as head.

movies receive only a few. Models trained only to maximize ranking accuracy may overfit to popular items because these items appear more frequently in the training graph and in sampled training triples. To analyze this effect, item popularity is computed only from the training graph. For each item i , its degree is

$$d_i = |\{u \in \mathcal{U} : (u, i) \in \mathcal{E}_{\text{train}}\}|. \quad (7)$$

The use of training-only degrees avoids leaking validation or test information into popularity statistics. Items are divided into three popularity groups using the 50th and 80th percentiles of training degree: the bottom 50% are labeled as tail items, the next 30% as middle items, and the top 20% as head items. Equivalently, if p_{50} and p_{80} denote the two percentile cutoffs, then

$$g(i) = \begin{cases} \text{Tail}, & d_i < p_{50}, \\ \text{Middle}, & p_{50} \leq d_i < p_{80}, \\ \text{Head}, & d_i \geq p_{80}. \end{cases} \quad (8)$$

In our processed data, this produces 1,755 tail items; 1,071 middle items and 707 head items. Figure 2 shows the resulting group sizes.

These groups are used in two ways. First, they provide a diagnostic tool for measuring whether recommendations are concentrated on head items or distributed across the catalog. Second, they support the popularity-aware training objective used later in the project, where item groups can be used to compare and balance losses across head, middle, and tail movies. This makes the graph learning task not only an accuracy-oriented recommendation problem, but also a long-tail recommendation problem.

The resulting head, middle, and tail groups are used both by the proposed popularity-aware training components and by the evaluation protocol. In addition to standard ranking accuracy, we evaluate whether recommendation lists retrieve tail items, cover a larger portion of the catalog, and reduce their concentration on highly popular movies. The complete evaluation protocol is described in Section 5.2.

4 Methodology

This section describes the recommendation models implemented in this project. We first introduce three baselines: MostPopular, BPR matrix factorization, and LightGCN. These models represent increasing levels of modeling capacity, from non-personalized popularity-based ranking to personalized matrix factorization and graph-based collaborative filtering. We then describe the proposed

popularity-aware LightGCN variants, which extend the LightGCN backbone with loss equalization, degree-aware graph augmentation, contrastive regularization, and popularity-aware negative sampling to reduce popularity bias during training.

4.1 MostPopular baseline

The MostPopular baseline is a simple non-personalized recommender that ranks movies only by their global popularity in the training set. Although it does not learn user-specific preferences, it is an important baseline for this project because it directly reflects the popularity signal present in the data. If a model performs similarly to MostPopular, this may indicate that the model relies heavily on item popularity rather than personalized collaborative signals.

For each movie ($i \in \mathcal{I}$), we compute its training degree as

$$d_i = |\{u \in \mathcal{U} : (u, i) \in \mathcal{E}_{\text{train}}\}|, \quad (9)$$

where d_i is the number of positive training interactions connected to movie i . The score assigned by the MostPopular baseline is then defined as

$$s_{\text{MP}}(u, i) = d_i. \quad (10)$$

Since the score does not depend on the user u , all users receive the same global ranking before training-item masking is applied.

At evaluation time, movies already interacted with by the user in the training set are excluded from the candidate list. The remaining movies are ranked by $s_{\text{MP}}(u, i)$, and the top- K movies are returned as recommendations. This baseline provides a useful lower-complexity reference point: it helps determine whether more advanced models improve personalization and long-tail recommendation behavior beyond simply recommending the most frequently observed movies.

4.2 BPR matrix factorization

Bayesian Personalized Ranking Matrix Factorization (BPR-MF) is employed as a personalized collaborative filtering baseline for implicit-feedback recommendation. Unlike point-wise approaches that attempt to predict an absolute interaction value, BPR-MF learns the relative preference of a user between an observed item and an unobserved item. This formulation is directly aligned with the top- K recommendation objective, in which relevant items should be ranked above irrelevant candidates.

Let $\mathcal{U}$ and $\mathcal{I}$ denote the sets of users and items, respectively. The implicit user-item interaction matrix is defined as

$$\mathbf{R} \in \{0, 1\}^{|\mathcal{U}| \times |\mathcal{I}|}, \quad (11)$$

where $R_{ui} = 1$ indicates that user u has interacted with item i , while $R_{ui} = 0$ denotes an unobserved interaction. Since an unobserved interaction does not necessarily imply that the user dislikes the item, BPR-MF treats the recommendation problem as pairwise preference learning rather than binary classification.

Each user u and item i is represented by a trainable latent embedding vector,

$$\mathbf{p}_u \in \mathbb{R}^d, \quad \mathbf{q}_i \in \mathbb{R}^d, \quad (12)$$

where d is the latent embedding dimension. The predicted preference score of user u for item i is computed using the inner product

$$\hat{y}_{ui} = \mathbf{p}_u^\top \mathbf{q}_i. \quad (13)$$

A larger value of $\hat{y}_{ui}$ indicates a stronger predicted preference.

During training, the model samples triplets (u, i, j) , where i is an item previously interacted with by user u and j is an unobserved item. The learning objective is to enforce

$$\hat{y}_{ui} > \hat{y}_{uj}. \quad (14)$$

The pairwise score difference is therefore defined as

$$\hat{x}_{uij} = \hat{y}_{ui} - \hat{y}_{uj} = \mathbf{p}_u^\top (\mathbf{q}_i - \mathbf{q}_j). \quad (15)$$

The model parameters are optimized using the BPR loss

$$\mathcal{L}_{\text{BPR}} = - \sum_{(u,i,j) \in \mathcal{D}_S} \ln \sigma(\hat{y}_{ui} - \hat{y}_{uj}) + \lambda (\|\mathbf{p}_u\|_2^2 + \|\mathbf{q}_i\|_2^2 + \|\mathbf{q}_j\|_2^2), \quad (16)$$

where $\mathcal{D}_S$ denotes the set of sampled training triplets, $\sigma(\cdot)$ is the sigmoid function, and λ is the regularization coefficient. The ranking term penalizes cases in which an unobserved item receives a higher score than an observed item, whereas the regularization term constrains the magnitude of the latent embeddings and reduces overfitting.

The user and item embeddings are learned using stochastic gradient-based optimization. For each sampled triplet, the model increases the compatibility between the user embedding and the positive-item embedding while decreasing the compatibility between the user embedding and the negative-item embedding. Consequently, items that are consistent with the user’s historical preference are expected to obtain higher ranking scores.

At inference time, the score $\hat{y}_{ui}$ is calculated for every candidate item. Items that already appear in the user’s training interactions are excluded to prevent the model from simply recommending previously observed items. The remaining items are sorted in descending order of their predicted scores, and the highest-ranked K items are returned as the recommendation list.

BPR-MF provides a useful personalized baseline because it captures user-specific preference patterns while remaining computationally efficient. However, it learns each user and item representation directly and does not explicitly propagate information through the user–item interaction graph. Therefore, it cannot directly model higher-order collaborative relationships, such as similarities between users connected through common items. This limitation motivates the use of graph-based recommendation models such as LightGCN.

4.3 LightGCN

Light Graph Convolutional Network (LightGCN) is employed as the graph-based collaborative filtering baseline in this study. In contrast to BPR-MF, which learns user and item embeddings independently, LightGCN explicitly represents user–item interactions as a bipartite graph and propagates embedding information through multiple graph convolution layers. This allows the model to capture higher-order collaborative relationships beyond direct user–item interactions.

Let

$$\mathcal{G} = (\mathcal{U}, \mathcal{I}, \mathcal{E}) \quad (17)$$

denote the user–item interaction graph, where $\mathcal{U}$ is the set of users, $\mathcal{I}$ is the set of items, and $\mathcal{E}$ is the set of observed interactions. An edge $(u, i) \in \mathcal{E}$ exists if user u has interacted with item i .

Given the implicit interaction matrix

$$\mathbf{R} \in \{0, 1\}^{|\mathcal{U}| \times |\mathcal{I}|}, \quad (18)$$

the adjacency matrix of the bipartite graph is constructed as

$$\mathbf{A} = \begin{bmatrix} \mathbf{0} & \mathbf{R} \\ \mathbf{R}^\top & \mathbf{0} \end{bmatrix}. \quad (19)$$

The corresponding degree matrix is

$$\mathbf{D}_{vv} = \sum_t A_{vt}, \quad (20)$$

and the symmetrically normalized adjacency matrix is defined as

$$\hat{\mathbf{A}} = \mathbf{D}^{-\frac{1}{2}} \mathbf{A} \mathbf{D}^{-\frac{1}{2}}. \quad (21)$$

Each user and item is associated with an initial trainable embedding. The complete initial embedding matrix is written as

$$\mathbf{E}^{(0)} = \begin{bmatrix} \mathbf{E}_U^{(0)} \\ \mathbf{E}_I^{(0)} \end{bmatrix} \in \mathbb{R}^{(|\mathcal{U}| + |\mathcal{I}|) \times d}, \quad (22)$$

where d is the embedding dimension. Unlike conventional graph convolutional networks, LightGCN removes feature transformation matrices, nonlinear activation functions, and bias terms. The embedding propagation rule is therefore simplified to

$$\mathbf{E}^{(l+1)} = \hat{\mathbf{A}}\mathbf{E}^{(l)}, \quad l = 0, 1, \dots, L-1, \quad (23)$$

where L denotes the number of propagation layers.

At the node level, the representation of user u at layer $l+1$ is computed as

$$\mathbf{e}_u^{(l+1)} = \sum_{i \in \mathcal{N}_u} \frac{1}{\sqrt{|\mathcal{N}_u||\mathcal{N}_i|}} \mathbf{e}_i^{(l)}, \quad (24)$$

where $\mathcal{N}_u$ denotes the set of items connected to user u . Similarly, the item representation is updated as

$$\mathbf{e}_i^{(l+1)} = \sum_{u \in \mathcal{N}_i} \frac{1}{\sqrt{|\mathcal{N}_i||\mathcal{N}_u|}} \mathbf{e}_u^{(l)}. \quad (25)$$

The normalization coefficient prevents high-degree nodes from dominating the aggregation process solely because they have more neighbors.

Each propagation layer captures collaborative information at a different graph distance. The initial embedding $\mathbf{E}^{(0)}$ represents node-specific latent factors. The first propagation layer aggregates information from directly connected neighbors, while deeper layers incorporate signals from higher-order paths such as

$$u \rightarrow i \rightarrow u' \rightarrow i'. \quad (26)$$

Such paths allow the model to infer that an unobserved item may be relevant to a user when it is connected through users with similar interaction patterns.

Instead of using only the output of the final propagation layer, LightGCN combines the embeddings produced at all layers. The final embedding is defined as

$$\mathbf{E}^* = \sum_{l=0}^L \alpha_l \mathbf{E}^{(l)}, \quad (27)$$

subject to

$$\sum_{l=0}^L \alpha_l = 1, \quad \alpha_l \geq 0. \quad (28)$$

In the standard formulation, uniform layer weights are commonly used:

$$\alpha_l = \frac{1}{L+1}. \quad (29)$$

Accordingly, the final user and item embeddings are

$$\mathbf{e}_u^* = \sum_{l=0}^L \alpha_l \mathbf{e}_u^{(l)}, \quad (30)$$

and

$$\mathbf{e}_i^* = \sum_{l=0}^L \alpha_l \mathbf{e}_i^{(l)}. \quad (31)$$

The predicted preference score between user u and item i is calculated by the inner product

$$\hat{y}_{ui} = (\mathbf{e}_u^*)^\top \mathbf{e}_i^*. \quad (32)$$

A higher score indicates that the item is expected to be more relevant to the user.

LightGCN is trained using the same pairwise ranking principle as BPR-MF. For each sampled triplet (u, i, j) , where i is an observed positive item and j is an unobserved item, the model minimizes

$$\mathcal{L}_{\text{LightGCN}} = - \sum_{(u,i,j) \in \mathcal{D}_S} \ln \sigma(\hat{y}_{ui} - \hat{y}_{uj}) + \lambda \left\| \mathbf{E}^{(0)} \right\|_F^2, \quad (33)$$

where $\mathcal{D}_S$ is the set of sampled training triplets, $\sigma(\cdot)$ is the sigmoid function, and λ controls the regularization strength. In the standard LightGCN formulation, regularization is applied to the initial trainable embeddings rather than separately to the propagated embeddings, since the latter are deterministically obtained from $\mathbf{E}^{(0)}$.

During optimization, the gradient of the ranking loss is back-propagated through all graph propagation layers to update the initial user and item embeddings. Consequently, a training triplet may influence not only the directly involved user and items but also their connected neighbors through the graph structure.

At inference time, the final user embedding is compared with the final embedding of every candidate item. Items already observed in the training set are excluded, and the remaining items are sorted in descending order according to $\hat{y}_{ui}$. The top- K highest-scoring items are returned as the personalized recommendation list.

Compared with BPR-MF, LightGCN is able to exploit higher-order collaborative signals through neighborhood propagation while preserving a relatively simple model architecture. The removal of feature transformations and nonlinear activations reduces the number of model parameters and avoids introducing operations that are not necessarily beneficial for collaborative filtering. However, the propagation mechanism also makes LightGCN sensitive to the degree distribution of the interaction graph. Popular items have substantially more neighbors, participate in more propagation paths, and may influence a larger number of user representations. As a result, their embeddings can become disproportionately dominant in the ranking process, which motivates the popularity-aware modifications introduced in the proposed method.

4.4 Proposed Method

Graph-based collaborative filtering models such as LightGCN [2] are particularly susceptible to popularity bias: because the neighborhood-aggregation step propagates a signal proportional to node degree, high-degree (popular) item nodes receive disproportionately more gradient updates than low-degree (long-tail) item nodes. Over training, this drives popular-item embeddings to larger norms and causes them to dominate the dot-product ranking for almost every user, irrespective of individual user preference.

We propose *Popularity-Aware LightGCN*, which augments the LightGCN backbone with four complementary interventions. We introduce them in the order below because each one closes a specific gap left open by the ones before it, rather than as four independent, parallel tricks; all four are independently switchable, so that disabling all of them recovers plain LightGCN trained with uniform negative sampling:

- (1) an **objective-level** re-weighting of the loss, *Item Loss Equalization* (Section 4.4.1), which reshapes per-interaction gradients but leaves the propagation graph itself untouched;
- (2) a **structure-level** perturbation of the propagation graph, *Degree-Aware Graph Augmentation* (Section 4.4.2), which weakens the structural dominance of popular items during message passing but does not directly shape the resulting embedding geometry;
- (3) a **representation-level** contrastive regularizer computed across two augmented views of the graph (Section 4.4.3), which explicitly encourages a well-spread embedding geometry but leaves the negative sampling distribution untouched;
- (4) a **sampling-level** bias in the negative distribution, *Popularity-Aware Negative Sampling* (Section 4.4.4), which closes this last gap by directly and repeatedly challenging popular items during optimization.

All four components are implemented and independently switchable (Section 5.3); however, the configurations reported in Section 6 sweep components (1), (3), and (4), while component (2) is exercised only in its role as the view-construction mechanism for component (3), rather than as a stand-alone main-pass regularizer — see the note accompanying Table 5 for details. Section 4.4.5 combines the four components into a single training objective and summarizes the full training procedure, and Section 4.4.6 describes the protocol used to select and evaluate the final model without leaking test-set information into training decisions.

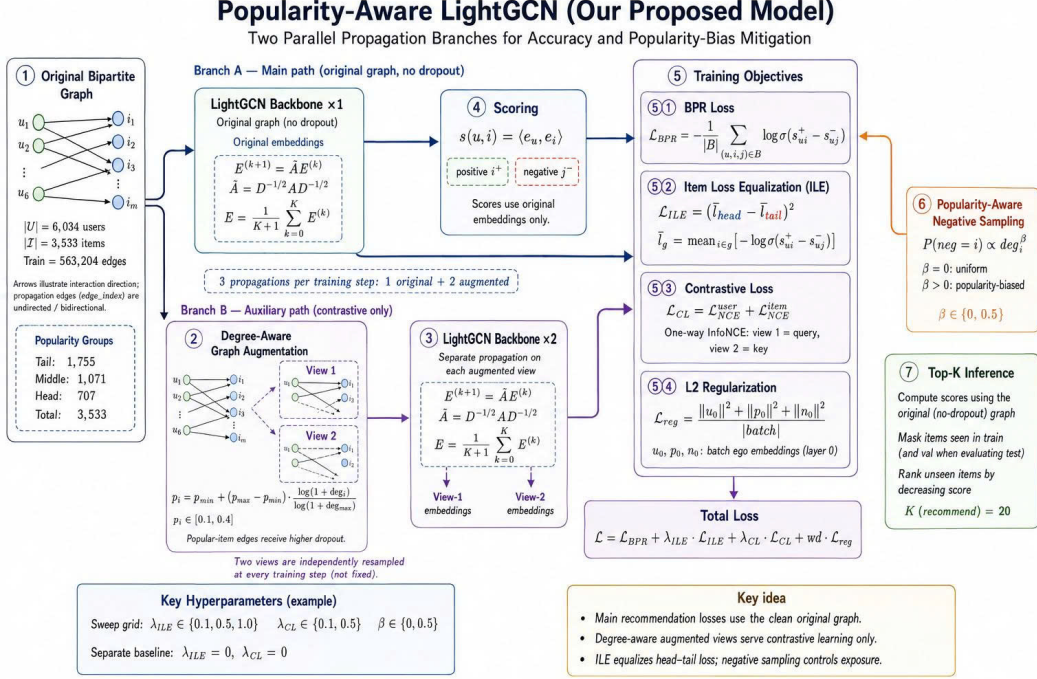


Figure 3: Overview of the proposed Popularity-Aware LightGCN framework. The architecture consists of two parallel propagation branches: (i) the main recommendation branch operating on the original user–item interaction graph to preserve recommendation accuracy, and (ii) an auxiliary contrastive learning branch that employs degree-aware graph augmentation to mitigate popularity bias. The model is jointly optimized using Bayesian Personalized Ranking (BPR) loss, Item Loss Equalization (ILE), contrastive loss, and ℓ_2 regularization, while popularity-aware negative sampling further improves representation learning. During inference, only the original graph is used to generate Top- K recommendations, ensuring inference efficiency comparable to the original LightGCN.

4.4.1 Backbone and Item Loss Equalization

We first describe the LightGCN backbone and its base training objective, which every subsequent component in this section builds upon, before introducing our first bias-mitigation component, Item Loss Equalization.

Backbone. Let $\mathcal{U}$ and $\mathcal{I}$ denote the sets of users and items, and let $e_u^{(0)}, e_i^{(0)} \in \mathbb{R}^d$ be their layer-0 (ego) embeddings, held in a single embedding table indexed over $\mathcal{U} \cup \mathcal{I}$. Following LightGCN, we propagate embeddings over the user–item bipartite graph using the symmetric normalized adjacency $\tilde{A} = D^{-1/2} A D^{-1/2}$, without feature transformation or non-linear activation:

$$E^{(k+1)} = \tilde{A} E^{(k)}, \quad E = \frac{1}{K+1} \sum_{k=0}^K E^{(k)}, \quad (34)$$

where $E^{(k)}$ stacks the embeddings of all nodes at layer k and K is the number of propagation layers. The final user and item embeddings e_u, e_i are read off from E , and the predicted preference score is their inner product, $s(u, i) = e_u^\top e_i$.

Base objective. Positive pairs (u, i^+) are sampled uniformly from the set of training interactions, and the backbone is trained with the Bayesian Personalized Ranking (BPR) loss [4] over the resulting triplets (u, i^+, i^-) , where i^- is a sampled negative (Section 4.4.4 describes our negative sampler):

$$\ell(u, i^+, i^-) = -\log \sigma(s(u, i^+) - s(u, i^-)), \quad \mathcal{L}_{BPR} = \frac{1}{|B|} \sum_{(u, i^+, i^-) \in B} \ell(u, i^+, i^-), \quad (35)$$

together with an ℓ_2 penalty on the ego embeddings of the nodes present in the batch, averaged over the batch:

$$\mathcal{L}_{reg} = \frac{1}{|\mathcal{B}|} \sum_{(u, i^+, i^-) \in \mathcal{B}} \left(\|e_u^{(0)}\|_2^2 + \|e_{i^+}^{(0)}\|_2^2 + \|e_{i^-}^{(0)}\|_2^2 \right), \quad (36)$$

weighted by λ_{reg} in the total objective (Eq. (43)).

Item Loss Equalization (ILE). To counteract popularity bias directly at the objective level, we partition the item catalog into popularity groups — *head*, *middle*, *tail* — based on percentiles of item degree computed on the training graph (tail: below the 50th percentile; head: above the 80th percentile). For a training mini-batch $\mathcal{B}$, let $\mathcal{B}_{head}$ and $\mathcal{B}_{tail}$ be the subsets of triplets whose positive item i^+ falls in the head and tail groups, respectively (triplets whose i^+ falls in the middle group belong to neither subset and do not contribute to $\mathcal{L}_{ILE}$), and let

$$\bar{\ell}_g = \frac{1}{|\mathcal{B}_g|} \sum_{(u, i^+, i^-) \in \mathcal{B}_g} \ell(u, i^+, i^-), \quad g \in \{head, tail\}, \quad (37)$$

be the mean BPR loss within each group (the penalty is only evaluated when both groups are non-empty in the batch; otherwise it is defined as zero). We define the Item Loss Equalization penalty as the squared discrepancy between the two group losses,

$$\mathcal{L}_{ILE} = (\bar{\ell}_{head} - \bar{\ell}_{tail})^2. \quad (38)$$

Because tail items receive fewer positive interactions during training, $\bar{\ell}_{tail}$ is typically larger than $\bar{\ell}_{head}$ under plain BPR training; minimizing Eq. (38) therefore pulls $\bar{\ell}_{tail}$ toward $\bar{\ell}_{head}$, effectively re-weighting the optimization in favor of tail interactions. The squared, symmetric formulation is lower-bounded by zero and admits no incentive to instead *increase* $\bar{\ell}_{tail}$ to close the gap — a degenerate failure mode we observed with the naive, unsigned difference $\bar{\ell}_{head} - \bar{\ell}_{tail}$, which collapses the model toward a popularity-only ranking as its weight increases. ILE is a purely objective-level intervention: it reshapes the gradient contributed by each interaction according to the popularity of its positive item but leaves the propagation graph in Eq. (34) completely unchanged, isolating its effect from the structural intervention introduced next.

4.4.2 Degree-Aware Graph Augmentation

ILE re-weights the training objective but does not address the structural cause of popularity bias: high-degree item nodes still dominate neighborhood aggregation in Eq. (34). We introduce an optional, complementary intervention that perturbs the propagation graph itself, so that popular items contribute a proportionally smaller share of the aggregated signal during training.

Degree-aware edge dropout. For each item i with training-graph degree d_i , we define a drop probability that grows logarithmically with popularity,

$$p_i = p_{min} + (p_{max} - p_{min}) \cdot \frac{\log(1 + d_i)}{\log(1 + d_{max})}, \quad p_i \in [p_{min}, p_{max}], \quad (39)$$

where $d_{max} = \max_i d_i$ and p_{min}, p_{max} are hyperparameters. Popular items are thus dropped from the propagation graph more aggressively than long-tail items.

Symmetric dropout scheme. The bipartite graph is stored as an undirected edge list, i.e., every training interaction (u, i) contributes both a $u \rightarrow i$ and an $i \rightarrow u$ edge. To preserve the symmetry required by the normalized adjacency $\tilde{A}$, the drop decision is sampled once per interaction — retaining the $u \rightarrow i$ edge independently with probability $1 - p_i$ — and mirrored onto the reverse $i \rightarrow u$ edge, rather than drawing the two directions independently. This yields an augmented adjacency $\hat{A}$ that remains a valid undirected graph. As an ablation control, we also implement *uniform* edge dropout, which drops every edge independently with a constant probability regardless of item degree, isolating the contribution of the degree-aware weighting in Eq. (39) from that of dropout regularization in general. We refer to the degree-aware/uniform choice as the *dropout scheme*, reused identically by the contrastive branch in Section 4.4.3.

Usage. This augmentation is optional in two independent senses: (i) it can be toggled on or off as a stand-alone regularizer applied to the forward pass used to compute $\mathcal{L}_{BPR}$ and $\mathcal{L}_{ILE}$, and (ii) it is, independently of (i), also used to construct the two augmented views consumed by the contrastive objective described next. At inference time the augmentation is always disabled: item ranking is computed on the original, unperturbed graph, so the augmentation acts purely as a training-time regularizer. In the configurations reported in Section 6, usage (ii) is always active while usage (i) is not: the main forward pass is never itself augmented, so the structural intervention studied here is validated only through the contrastive branch of Section 4.4.3.

4.4.3 Contrastive Regularization across Augmented Views

Degree-aware augmentation (Section 4.4.2) weakens the structural dominance of popular items during message passing, but it does not directly control the geometry of the resulting embedding space: item embeddings can still collapse into a narrow region dominated by popular items even on an augmented graph. We therefore add a representation-level regularizer that explicitly targets this geometry. Following the self-supervised graph learning paradigm [6], we add an auxiliary contrastive objective that encourages the embedding space to remain well-spread rather than collapsing toward a small set of popular items. At each training step we sample two independent augmented graphs $\hat{A}^{(1)}, \hat{A}^{(2)}$ from the *original* graph A , using the same edge-dropout scheme introduced in Section 4.4.2 — degree-aware (Eq. (39)) in our main experiments, or uniform in the ablation control — independently of whether the main forward pass used for Eq. (35) is itself augmented. Each view is propagated through Eq. (34) to obtain node embeddings $E^{(1)}, E^{(2)}$, and the two views of the same node are treated as a positive pair, with all other nodes in the batch serving as negatives (InfoNCE):

$$\mathcal{L}_{NCE}(\mathcal{N}) = -\frac{1}{|\mathcal{N}|} \sum_{n \in \mathcal{N}} \log \frac{\exp(\hat{z}_n^{(1)\top} \hat{z}_n^{(2)} / \tau)}{\sum_{m \in \mathcal{N}} \exp(\hat{z}_n^{(1)\top} \hat{z}_m^{(2)} / \tau)}, \quad (40)$$

where $\hat{z}_n^{(v)} = E_n^{(v)} / \|E_n^{(v)}\|$ is the ℓ_2 -normalized embedding of node n in view v , and τ is a temperature hyperparameter. We apply Eq. (40) separately to the set of *unique* users and the set of unique positive items present in the mini-batch, $\mathcal{U}_B$ and $\mathcal{I}_B^+$, and sum the two terms:

$$\mathcal{L}_{CL} = \mathcal{L}_{NCE}(\mathcal{U}_B) + \mathcal{L}_{NCE}(\mathcal{I}_B^+). \quad (41)$$

The InfoNCE objective is known to jointly promote *alignment* of positive pairs and approximately *uniform* coverage of the embedding hypersphere [7]; the latter property directly discourages the embedding collapse toward a small set of popular items observed under plain BPR training, and is expected to increase catalog coverage.

4.4.4 Popularity-Aware Negative Sampling

The three components above reshape the training objective, the propagation graph, and the embedding geometry, respectively, but none of them alters *which* items are used as negatives during optimization: negatives are still drawn uniformly, so popular items are not disproportionately challenged even though they are precisely the items that most need to be pushed down in the ranking. We close this last gap with a sampling-level intervention. Standard BPR training draws the negative item i^- uniformly from the catalog. We instead sample negatives from a degree-biased categorical distribution,

$$i^- \sim \text{Categorical}(w), \quad w_i \propto d_i^\beta, \quad \beta \geq 0, \quad (42)$$

where β controls the sharpness of the bias toward popular items ($\beta = 0$ recovers uniform sampling). Sampled negatives that coincide with an item the user has already interacted with in training are rejected and re-drawn (subject to a bounded number of resampling attempts), preserving the standard implicit-BPR assumption that i^- is a non-interacted item. Because popular items are sampled as negatives far more frequently under Eq. (42), their score is discouraged more often and more strongly during training, directly counteracting their tendency to dominate the top- K ranking irrespective of true user preference.

4.4.5 Joint Training Objective

The full training objective for a mini-batch combines all four components,

$$\mathcal{L} = \mathcal{L}_{BPR} + \lambda_{reg} \mathcal{L}_{reg} + \lambda_{ILE} \mathcal{L}_{ILE} + \lambda_{CL} \mathcal{L}_{CL}, \quad (43)$$

Algorithm 1 One training epoch of Popularity-Aware LightGCN

Require: training graph A , item degrees $\{d_i\}$, popularity groups, weights $\lambda_{ILE}, \lambda_{CL}$, temperature τ , sampling sharpness β , switches for main-pass augmentation (AUG) and contrastive learning (CL)

- 1: **for** each mini-batch **do**
 - 2: Sample positive pairs (u, i^+) uniformly from the training interactions
 - 3: Sample negatives $i^- \sim \text{Categorical}(d^\beta)$ (Eq. (42)); reject and re-draw if i^- is a training positive of u
 - 4: $\hat{A}_{\text{main}} \leftarrow$ degree-aware (or uniform) dropout of A if AUG is enabled, else A
 - 5: Propagate $\hat{A}_{\text{main}}$ (Eq. (34)) to obtain $s(u, i^+)$, $s(u, i^-)$, and the ego embeddings of the batch
 - 6: Compute $\mathcal{L}_{BPR}$ (Eq. (35)) and $\mathcal{L}_{reg}$ (Eq. (36))
 - 7: **if** ILE is enabled **then**
 - 8: Partition the batch by the popularity group of i^+ ; compute $\mathcal{L}_{ILE}$ (Eq. (38))
 - 9: **end if**
 - 10: **if** CL is enabled **then**
 - 11: Sample two fresh views $\hat{A}^{(1)}, \hat{A}^{(2)}$ of the original graph A via the configured edge-dropout scheme; propagate both
 - 12: Compute $\mathcal{L}_{CL}$ on the batch’s unique users and positive items (Eq. (40))
 - 13: **end if**
 - 14: Assemble $\mathcal{L}$ (Eq. (43)); backpropagate and update parameters with Adam
 - 15: **end for**
 - 16: Every E epochs, evaluate Recall@ K on the validation split and retain the checkpoint with the highest validation Recall@ K ; stop early after P evaluations without improvement
-

where $\mathcal{L}_{BPR}$ (Eq. (35)) is computed with the popularity-aware negative sampler of Eq. (42), optionally over the augmented graph of Eq. (39); $\mathcal{L}_{ILE}$ reuses the same per-triplet losses from that forward pass, grouped by the popularity of i^+ (Eq. (38)); and $\mathcal{L}_{CL}$ is computed on two freshly sampled augmented views of the original graph (Eq. (40)). Each of λ_{ILE} , λ_{CL} , the negative-sampling sharpness β , and the two graph-side switches (main-pass augmentation and contrastive learning) can be set to zero independently. The ablation reported in Section 6 sweeps λ_{ILE} , λ_{CL} , and β , while keeping the main-pass augmentation switch off throughout (Table 5); a standalone ablation of main-pass augmentation is left for future work. At inference time, item ranking for a user u is computed with the score $s(u, i) = e_u^\top e_i$ evaluated on the original, unaugmented graph: neither the augmentation nor the contrastive branch is active at inference, so both act purely as training-time regularizers on an otherwise unmodified LightGCN scoring function. Algorithm 1 summarizes one training epoch.

4.4.6 Model Selection and Evaluation Protocol

Having described the model and its unified training objective, we conclude with the protocol used to select and evaluate the trained model, which underpins the validity of the results reported in Section 6. To obtain an unbiased estimate of generalization performance, the validation and test splits are strictly separated in role throughout training. Every E epochs, the current model is evaluated on the validation split and the checkpoint achieving the highest validation Recall@ K is retained; training stops early if no improvement is observed for P consecutive evaluations (Algorithm 1). The test split is touched *exactly once*, after training has concluded, using the retained best-validation checkpoint; at no point is a test-set metric used to make a training or model-selection decision. All test-set results reported in Section 6 therefore reflect genuine out-of-sample generalization rather than a checkpoint selected in hindsight on the test set. Unless otherwise noted, $K = 20$ throughout: Recall@20 is used both as the model-selection criterion in Algorithm 1 and as the primary reported accuracy metric in Section 6.

5 Experimental setup

All models share a single data pipeline, evaluation function, and hyperparameter configuration so that reported numbers are directly comparable. Preprocessing, splitting, graph construction, evaluation, and result logging are implemented once in `src/` and imported by every model, rather than being reimplemented per model.

5.1 Implementation details

We implement all models in PyTorch and load MovieLens 1M through the `MovieLens1M` interface of PyTorch Geometric. Every experiment begins by calling a shared `set_seed` routine that seeds Python, NumPy, and PyTorch ($\text{seed} = 42$), so runs are reproducible. Hyperparameters and paths are defined in a single `src/config.py` module and imported everywhere; no value is hardcoded inside individual models. Metric results are appended to a single `results/results.csv` through a shared `save_result` function with a fixed column schema, ensuring that all models are logged and compared identically.

Architecture and initialization. All models share an embedding dimension of $d = 64$. The backbone (Section 4.4.1) uses $K = 2$ propagation layers in our main experiments, which we found to avoid the over-smoothing effect observed with a deeper, 3-layer backbone on this dataset (Section 6). Embedding weights are initialized from $\mathcal{N}(0, 0.1^2)$.

Popularity groups. Item popularity groups (head/middle/tail, Section 4.4.1) are computed once from training-graph item degrees using the 50th and 80th percentile cutoffs described above, and held fixed throughout training and evaluation.

Optimization. All models are trained with Adam using a learning rate of 10^{-3} , an ℓ_2 weight of $\lambda_{reg} = 10^{-4}$ (Eq. (36)), and mini-batches of 4096 triplets. Gradients are clipped to a maximum global norm of 1.0 after every backward pass. Models are trained for up to 100 epochs under the early-stopping and checkpointing protocol of Section 4.4.6 ($E = 5$, $P = 10$).

The complete list of hyperparameters and their explored ranges is given in Section 5.3 (Table 4), together with the exact combination selected for each configuration reported in Section 6 (Table 5).

Software, hardware, and reproducibility. The model is implemented in PyTorch and trained on a single NVIDIA A100 GPU. Each reported configuration is trained with three random seeds (42, 0, 1); the model parameters and the mini-batch sampling stream are seeded separately, so that a given run is exactly reproducible. Each run periodically checkpoints its model and optimizer state, allowing training to be resumed after interruption without altering the final result, and logs per-epoch training and validation metrics for post-hoc inspection.

5.2 Evaluation protocol

We use per-user chronological splitting: for each user, the most recent positive interaction is held out for testing, the next most recent for validation, and the rest for training. Consequently every user has exactly one test item, which reduces $\text{Recall}@K$ to a hit indicator and $\text{NDCG}@K$ to $1/\log_2(\text{rank}+1)$ when the test item appears in the top- K list.

Because MovieLens 1M is small, we adopt *full-item ranking* rather than sampled-negative evaluation: for each user we score the entire catalogue, mask the items already seen in that user’s *training and validation* history, and take the top- K . The identical `evaluate_full_ranking` function is applied to every model, so no model is scored on a different candidate set than another. Alongside accuracy ($\text{Recall}@K$, $\text{NDCG}@K$) we report popularity-aware metrics: Tail $\text{Recall}@K$ (recall restricted to users whose test item is a tail item), catalogue coverage (fraction of items recommended to at least one user), average recommendation popularity (ARP, mean $\log(1 + \text{deg})$ over recommended slots), and head/middle/tail exposure. We evaluate at $K \in \{10, 20\}$ and report $K = 20$ as the primary cutoff.

Items are partitioned into popularity groups using *training-graph* degree only: head = top 20%, middle = next 30%, tail = bottom 50% (percentile cutoffs p_{50} and p_{80}). Figure 4 shows the resulting long-tailed degree distribution, which motivates the tail- and coverage-oriented metrics above.

5.3 Hyperparameters

Unless stated otherwise, all embedding-based models use the shared settings in Table 3. LightGCN and its variants use 2 propagation layers in the main experiments (with layer-wise embeddings averaged, including layer 0); a 3-layer backbone was also explored but over-smoothed on this dataset (see Table 4). The IL2E penalty weight λ_{IL2E} is swept over $\{0, 0.1, 0.5, 1\}$ in the ablation of Section 6.5.

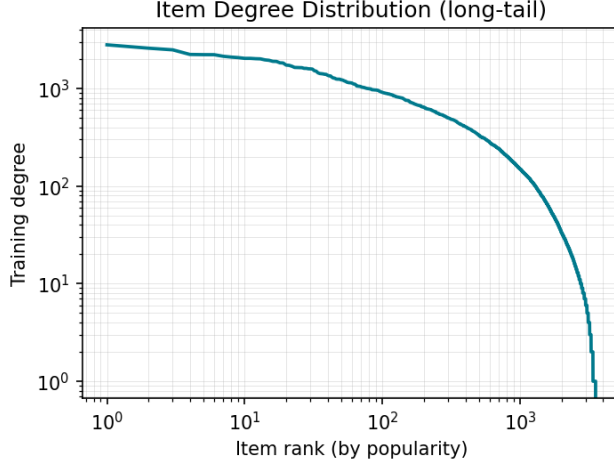


Figure 4: Item training-degree distribution on MovieLens 1M (log–log rank–frequency). A small number of popular movies accumulate most interactions while the majority are rarely interacted with, confirming the popularity bias the project targets.

Table 3: Shared hyperparameters (`src/config.py`).

Hyperparameter	Value
Embedding dimension	64
LightGCN layers (main / explored)	2 / {2, 3}
Learning rate	1×10^{-3}
Batch size	4096
Max epochs	100 (early stopping)
Weight decay (L_2)	1×10^{-4}
Negatives per positive	1
λ_{ILE} (default)	1.0
Top- K cutoffs	10, 20
Random seeds	42, 0, 1

Table 4 lists every hyperparameter introduced in Section 4.4 and Section 5.1, together with the value or range used in this work. Table 5 then gives the exact combination selected for each named configuration reported in Section 6.

6 Results and analysis

We first verify the evaluation pipeline with the non-personalized MostPopular baseline, then compare all models on accuracy and popularity-aware metrics, and finally isolate the contribution of each component through an ablation study. All numbers are produced by the shared full-ranking evaluation of Section 5.2; embedding-based models (LightGCN and the PopAware variants) are reported as the mean over three seeds (42, 0, 1) with the standard deviation.

6.1 Main results

Table 6 reports the primary metrics at $K = 20$. The five PopAware operating points correspond to the $(\lambda_{ILE}, \lambda_{CL}, \beta)$ combinations of Table 5; they are deliberately chosen to trace out the accuracy–diversity trade-off rather than to win every column.

The MostPopular results exhibit the expected sanity-check signature and confirm the evaluation pipeline is correct: accuracy is low ($\text{Recall@20} = 0.072$, $\text{NDCG@20} = 0.027$), tail recall is exactly zero, catalogue coverage is only 5.0% (every user receives almost the same list of popular movies), and head-group exposure is 100% (tail and middle exposure are both 0). A high ARP of 7.53 further

Table 4: All hyperparameters of Popularity-Aware LightGCN.

Symbol	Description	Value / range
d	Embedding dimension	64
K	Propagation layers (Eq. (34))	$\{2, 3\}$
—	Adam learning rate	10^{-3}
λ_{reg}	ℓ_2 weight (Eq. (36))	10^{-4}
—	Mini-batch size	4096 triplets
—	Gradient-clipping max norm	1.0
—	Maximum training epochs	100
E	Validation frequency (Algorithm 1)	every 5 epochs
P	Early-stopping patience (Algorithm 1)	10 evaluations
—	Head / tail popularity percentile cutoffs	80th / 50th
λ_{ILE}	ILE weight (Eq. (43))	$\{0, 0.1, 0.5, 1.0\}$
λ_{CL}	Contrastive weight (Eq. (43))	$\{0, 0.1, 0.5\}$
τ	InfoNCE temperature (Eq. (40))	0.2
p_{min}, p_{max}	Degree-aware dropout range (Eq. (39))	0.1, 0.4
$p_{uniform}$	Uniform-dropout ablation probability	0.1
β	Negative-sampling sharpness (Eq. (42))	$\{0, 0.5\}$

Table 5: Exact hyperparameter combination for each configuration reported in Section 6 (test-set results, mean over 3 seeds: 42, 0, 1). All configurations use $d = 64$, $K = 2$, $\tau = 0.2$, and the degree-aware dropout scheme; the main forward pass used for $\mathcal{L}_{BPR}$ and $\mathcal{L}_{ILE}$ is *not* itself augmented in these results (aug_main = off for every row). Consequently, Degree-Aware Graph Augmentation (Section 4.4.2) is validated here only indirectly, through the view construction of the contrastive branch (Section 4.4.3); a standalone ablation of main-pass augmentation is left for future work.

Configuration	λ_{ILE}	λ_{CL}	β
LightGCN (baseline)	0	0	0
PopAware — accuracy	0.1	0.1	0.5
PopAware-BEST — balanced	1.0	0.1	0.5
PopAware — high-tail	1.0	0.1	0
PopAware — fairness	1.0	0.5	0

reflects the strong popularity concentration. Every personalized model improves substantially over this baseline, as expected.

6.2 Accuracy comparison

Plain LightGCN reaches Recall@20 = 0.1287 and NDCG@20 = 0.0497, essentially matching BPR-MF (0.1309 / 0.0511). Crucially, the popularity-aware training does *not* cost accuracy: PopAware-accuracy attains the *best* Recall@20 (0.1367) and NDCG@20 (0.0527) of all models, and both PopAware-high-tail and PopAware-BEST also exceed plain LightGCN on Recall and NDCG. Only the fairness-oriented point, which uses a strong contrastive weight ($\lambda_{CL} = 0.5$), trades measurable accuracy (Recall@20 = 0.105) for diversity. This answers the central question of Section 4.4: the Item Loss Equalization penalty and the degree-aware contrastive branch can be added to LightGCN while *preserving—and at moderate strength even slightly improving—*overall ranking accuracy.

6.3 Long-tail recommendation analysis

The value of the method is in the accuracy–diversity trade-off, visualised in Figure 5 and Figure 6. Plain LightGCN is markedly more biased than even BPR-MF: it collapses catalogue coverage to 39.8% (vs. 62.8% for BPR-MF) and recommends tail items to almost no one (Tail Recall@20 = 0.005). This is the over-concentration that message passing induces—high-degree head movies participate in many propagation paths and dominate the learned embeddings.

Table 6: Main results at $K = 20$ on MovieLens 1M (6,034 users, 3,533 items). Higher is better for Recall, NDCG, Tail Recall, and Coverage; lower ARP indicates less popularity bias. Embedding-based rows are mean $\pm$ std over seeds $\{42, 0, 1\}$. MostPopular is a sanity-check baseline.

Model	Recall@20	NDCG@20	Tail Recall@20	Coverage@20	ARP@20
MostPopular	0.0721	0.0272	0.0000	0.0498	7.53
BPR-MF	0.1309	0.0511	0.0109	0.6281	6.52
LightGCN	$0.1287 \pm .0005$	$0.0497 \pm .0002$	$0.0050 \pm .0009$	$0.3978 \pm .0043$	6.83
PopAware — accuracy	$0.1367 \pm .0035$	$0.0527 \pm .0006$	$0.0143 \pm .0023$	$0.4619 \pm .0289$	6.50
PopAware — high-tail	$0.1352 \pm .0020$	$0.0524 \pm .0004$	$0.0399 \pm .0023$	$0.4990 \pm .0099$	6.56
PopAware-BEST	$0.1338 \pm .0030$	$0.0518 \pm .0004$	$0.0324 \pm .0049$	$0.5384 \pm .0293$	6.40
PopAware — fairness	$0.1052 \pm .0039$	$0.0415 \pm .0011$	$0.0274 \pm .0035$	$0.7135 \pm .0027$	6.14

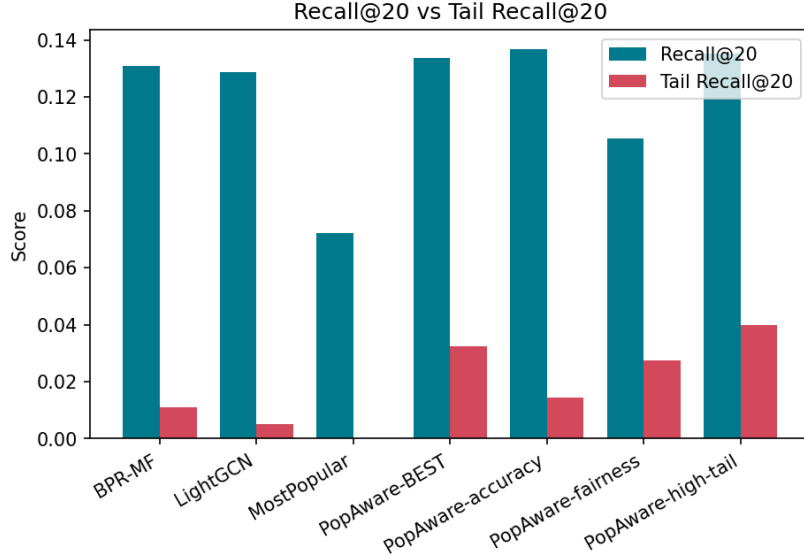


Figure 5: Recall@20 (accuracy) versus Tail Recall@20 (long-tail quality) per model. The PopAware variants lift tail recall by up to $8\times$ over LightGCN while keeping overall recall at or above the LightGCN level.

The PopAware variants correct this along a controllable curve. At the tail-oriented point, Tail Recall@20 rises from 0.005 to **0.040**—roughly an **eight-fold** increase—and coverage from 39.8% to 49.9%, while Recall@20 actually *increases* to 0.135. In other words, most of the long-tail improvement is obtained at no accuracy cost. Pushing further toward fairness raises coverage to **71.4%** and lowers ARP to the best value (6.14), at the expected price of a lower Recall@20 (0.105). PopAware-BEST sits in between (coverage 53.8%, Tail Recall 0.032, Recall 0.134) and is our recommended balanced operating point.

6.4 Exposure analysis

Figure 7 decomposes the top-20 recommendation slots into head/middle/tail exposure. MostPopular allocates 100% of slots to head items; plain LightGCN is nearly as skewed (95.9% head, 0.15% tail). The PopAware variants redistribute exposure away from the head in a graded manner: head exposure falls to 91.2% (high-tail) and 83.3% (fairness), with the freed slots going to middle and tail items (tail exposure rises from 0.15% to 3.6–4.8%). The three exposure fractions sum to 1 by construction, so the figure directly reads as a redistribution of a fixed slot budget.

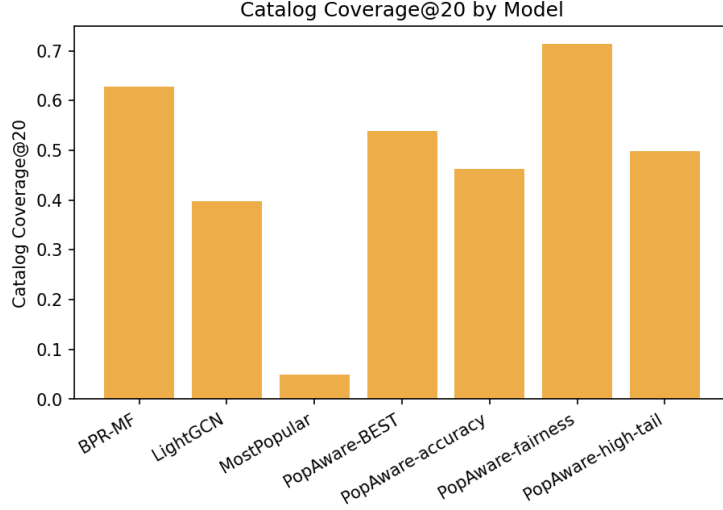


Figure 6: Catalogue Coverage@20 per model. LightGCN is the most concentrated of the personalized models; the PopAware variants progressively restore coverage, up to 71% at the fairness operating point.

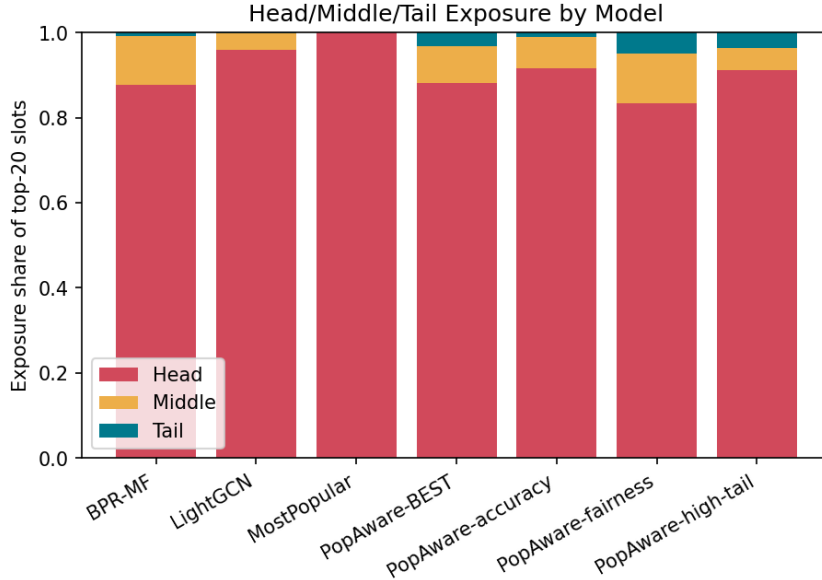


Figure 7: Head/middle/tail exposure of the top-20 slots per model (stacked to 1). Plain LightGCN devotes 95.9% of slots to head items; the PopAware variants progressively transfer exposure to middle and tail items.

6.5 Ablation study

Effect of λ_{ILE} . Table 7 sweeps the ILE weight with the other components fixed ($\lambda_{CL} = 0.1$, $\beta = 0$, $K = 2$). Increasing λ_{ILE} monotonically raises Tail Recall (from 0.006 to 0.037) and coverage (from 39% to 50%) while Recall@20 stays essentially flat around 0.135. This confirms the intuition of Section 4.4: ILE does not directly force tail recommendations, it only equalises per-group loss, and λ_{ILE} acts as a clean dial along the trade-off.

Effect of the contrastive weight λ_{CL} . Comparing the operating points, raising λ_{CL} from 0.1 to 0.5 (fairness point) is what pushes coverage to its maximum (71%) and ARP to its minimum (6.14),

Table 7: Sensitivity to the ILE weight λ_{ILE} ($\lambda_{\text{CL}} = 0.1$, $\beta = 0$, $K = 2$; single seed). Higher λ_{ILE} improves tail recall and coverage at nearly constant Recall@20.

λ_{ILE}	Recall@20	NDCG@20	Tail Recall@20	Coverage@20
0 (LightGCN)	0.1279	0.0496	0.0056	0.392
0.1	0.1356	0.0534	0.0131	0.444
0.5	0.1352	0.0526	0.0262	0.498
1.0	0.1354	0.0527	0.0374	0.503

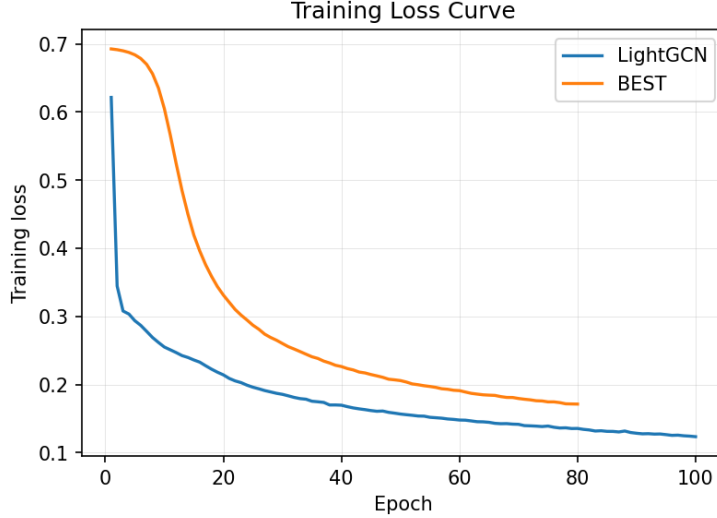


Figure 8: Per-epoch BPR training loss. The PopAware variants converge more slowly than plain LightGCN due to the ILE/contrastive regularisation but reach a comparable or lower final loss; model selection uses the best validation checkpoint.

at the cost of accuracy. The degree-aware contrastive branch is therefore the strongest diversity lever, whereas ILE contributes accuracy-neutral tail gains; the two are complementary, and PopAware-BEST combines a moderate ILE weight with a light contrastive weight to keep accuracy intact.

Convergence behaviour. Figure 8 shows the per-epoch BPR training loss. LightGCN minimises the ranking loss very quickly, whereas the PopAware variants descend more slowly because the ILE and contrastive terms regularise the early optimisation; they nonetheless reach an equal or lower BPR loss by convergence. To avoid unfairly penalising the slower-converging variants, all models use validation-based early stopping and the best validation checkpoint rather than a fixed final epoch.

6.6 Discussion

Taken together, the results support a clear, verifiable conclusion. On the MovieLens 1M user-movie graph, the popularity bias amplified by LightGCN can be *substantially reduced*—tail recall up to $8\times$, coverage improved by 25–79% relative, and ARP reduced—without sacrificing, and at moderate strength even slightly improving, Recall@20 and NDCG@20. The method exposes a single, well-behaved trade-off controlled by λ_{ILE} and λ_{CL} , so a practitioner can select an operating point (accuracy-first, balanced, or fairness-first) to match deployment goals. The main limitation is that with only 535 tail test items (8.9% of users), Tail Recall carries non-trivial variance; we mitigate this by averaging over three seeds and by also reporting the more stable coverage, exposure, and ARP metrics, which agree on the same conclusion.

7 Conclusion

This project investigated popularity bias in graph-based movie recommendation using the MovieLens 1M dataset. We formulated the task as implicit-feedback top- K recommendation on a user-movie bipartite graph and compared MostPopular, BPR-MF, LightGCN, and Popularity-Aware LightGCN under a shared full-ranking evaluation protocol. The proposed method combines Item Loss Equalization, degree-aware graph augmentation, contrastive regularization, and popularity-aware negative sampling. Compared with LightGCN, the balanced configuration increases TailRecall@20 by approximately $6.5\times$ and Catalog Coverage@20 by about 35%, while slightly improving Recall@20. These results show that popularity concentration can be substantially reduced without sacrificing overall recommendation accuracy.

The main limitation is that the experiments were conducted on a single dataset and only a small proportion of users had tail test items, which increases the variance of TailRecall. In addition, the independent contribution of each popularity-aware component has not been fully isolated. Future work should evaluate the method on additional long-tail datasets and conduct more detailed component-level ablation studies.

8 Team member contributions

This project was completed collaboratively by a team of five members. The work was divided based on individual strengths to ensure efficiency and quality.

- **Nguyen Nam Hoang 20252758M:** Performed data preprocessing, implicit-feedback conversion, and chronological train/validation/test splitting; constructed the user-movie graph, computed item degrees, and analyzed item popularity groups.
- **Nguyen Quoc Hung 20252568M:** Developed the evaluation pipeline and MostPopular baseline; implemented training-item masking, full-ranking evaluation, recommendation metrics, and integration of evaluation outputs across subsequent models.
- **Bui Trung Kien 20252567M:** Implemented and trained the BPR-MF and LightGCN baselines; developed negative sampling, BPR loss, and model scoring components, and generated baseline results for comparison.
- **Nguyen Minh Thuong 20252280M:** Designed and implemented Popularity-Aware LightGCN with Item Loss Equalization, Degree-Aware Contrastive Learning, and popularity-aware negative sampling; conducted leakage-free training, hyperparameter tuning, 3-seed ablations, and baseline comparisons.
- **Nguyen Tuan Kiet 20252763M:** Built and maintained the shared experiment infrastructure that made all models directly comparable-the single-source `config.py`, the fixed result schema, and the pipeline consolidating every run into one `results.csv` and the main results table. Generated figures, and wrote the Results and Analysis and Conclusion sections with the accuracy-diversity trade-off analysis. Coordinated the group’s review process. Prepared presentation slides.

All members contributed to discussions, experiment design, writing and reviewing the final report and making presentation slides.

References

- [1] F. Maxwell Harper and Joseph A. Konstan. The movielens datasets: History and context. *ACM Transactions on Interactive Intelligent Systems*, 5(4):1–19, 2015. doi: 10.1145/2827872.
- [2] Xiangnan He, Kuan Deng, Xiang Wang, Xiang Li, Yongdong Zhang, and Meng Wang. Lightgc: Simplifying and powering graph convolution network for recommendation. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 639–648. ACM, 2020. doi: 10.1145/3397271.3401063.
- [3] Anastasiia Klimashevskaya, Dietmar Jannach, Mehdi Elahi, and Christoph Trattner. A survey on popularity bias in recommender systems. *arXiv preprint arXiv:2308.01118*, 2023.

- [4] Steffen Rendle, Christoph Freudenthaler, Zeno Gantner, and Lars Schmidt-Thieme. Bpr: Bayesian personalized ranking from implicit feedback. In *Proceedings of the Twenty-Fifth Conference on Uncertainty in Artificial Intelligence*, pages 452–461, 2009.
- [5] Juno Prent and Masoud Mansoury. Correcting for popularity bias in recommender systems via item loss equalization. *arXiv preprint arXiv:2410.04830*, 2024.
- [6] Jiancan Wu, Xiang Wang, Fuli Feng, Xiangnan He, Liang Chen, Jianxun Lian, and Xing Xie. Self-supervised graph learning for recommendation. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 726–735. ACM, 2021. doi: 10.1145/3404835.3462862.
- [7] Tongzhou Wang and Phillip Isola. Understanding contrastive representation learning through alignment and uniformity on the hypersphere. In *Proceedings of the 37th International Conference on Machine Learning*, pages 9929–9939, 2020.